Implementation Plan: N8N Hands-On Intermediate Assessment

Topic 3: Data Validation, Cleaning & Duplicate Prevention

Document Navigation

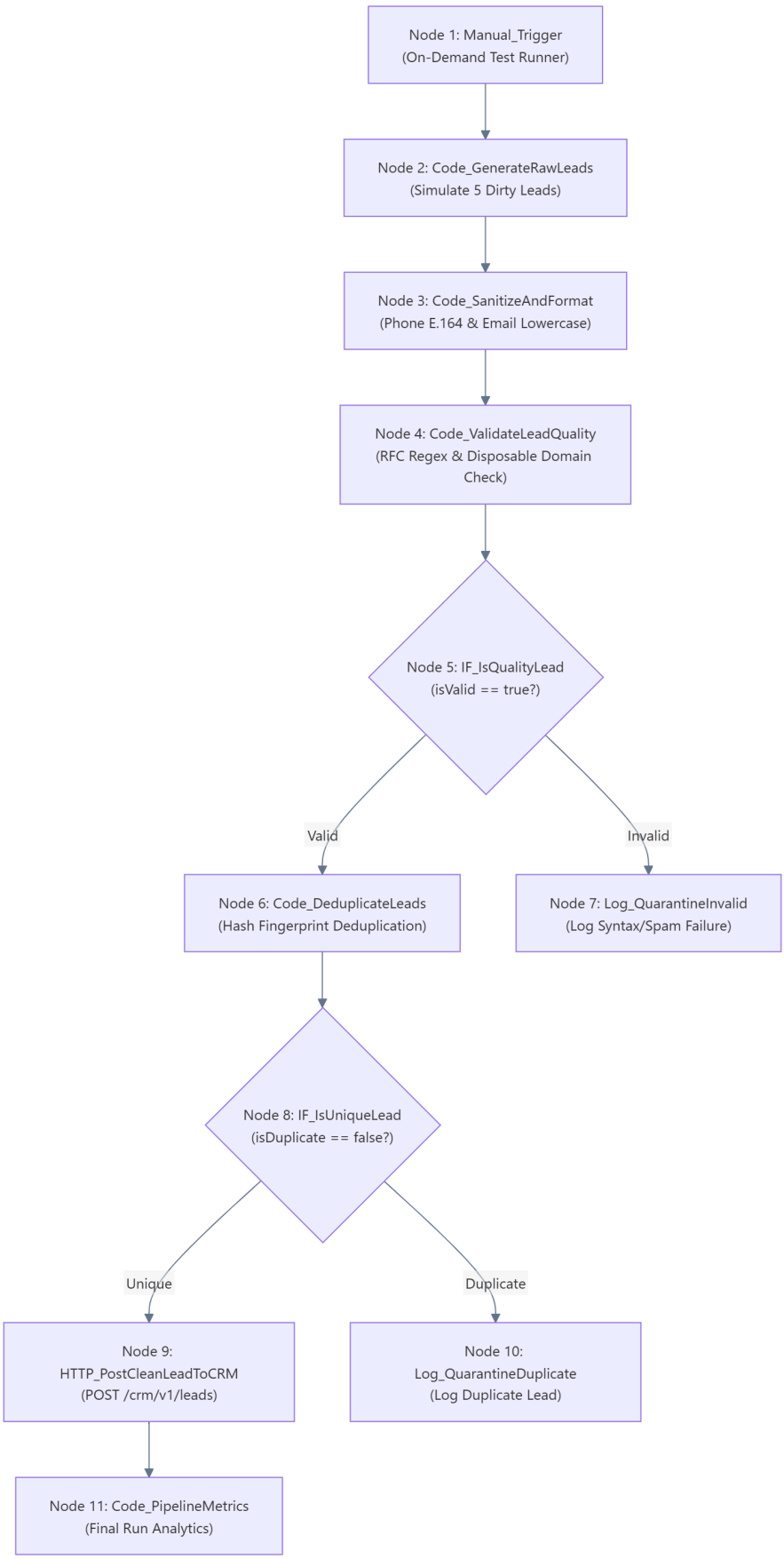
Concept Guide | Requirement Document | **Step-by-Step Implementation Plan** | Test Plan

1. Objective

Build an enterprise lead intake sanitization and deduplication pipeline in n8n. The workflow ingests raw, messy marketing leads, normalizes phone numbers into E.164, validates email structures, filters disposable domains, removes intra-batch duplicates, sends clean leads to a mock CRM, and routes bad leads to a quarantine log.

2. Canvas Architecture Overview

All rights reserved. Tayana Academy • Confidential Document



All rights reserved. Tayana Academy • Confidential Document

3. Prerequisites & Environment Setup

- [ ] n8n instance version 1.0.0 or higher.
- [ ] No external subscriptions or databases required.

---

4. Step-by-Step Practical Build Guide

Step 1: Mock CRM Lead Ingestion API Setup (Choose One of 3 Options)

To test lead ingestion without a paid HubSpot or Salesforce developer account:

Option A: Free Cloud Mock API via DummyJSON Custom Response (Recommended)

1. Visit **https://dummyjson.com/custom-response**.
2. In the JSON editor, paste:

```
{
  "crmLeadId": "lead_dummy_77391",
  "status": "CREATED",
  "crmPlatform": "HubSpot Simulated Sandbox",
  "message": "Lead ingested, phone verified in E.164 format, assigned to sales pipeline"
}
```

3. Set Status Code to `201 Created` and click **Generate API URL**.
4. Copy the generated URL (`https://dummyjson.com/c/your-id`) and paste into `HTTP_PostCleanLeadToCRM`.

Option B: Postman Mock Server

1. In Postman, add a request `POST /crm/v1/leads`.
2. Add an Example with Status Code `201 Created` and body:

```
{
  "crmLeadId": "lead_pstmn_3002",
  "status": "CREATED",
  "assignedRep": "Alex Morgan"
}
```

3. Click **Mock Collection** and copy your Postman mock URL: `https://{{mock-id}}.mock.pstmn.io/crm/v1/leads`.
4. Paste into `HTTP_PostCleanLeadToCRM`.

Option C: In-n8n Local Webhook Mock Server

1. Create a new workflow named `Mock CRM Server`.
2. Add a **Webhook Node** (`POST /mock/crm/leads`, Mode: Using Respond Node).

All rights reserved. Tayana Academy • Confidential Document

3. Connect a **Respond to Webhook Node**:
   - **Response Code**: `201`
   - **Response Body (Expression)**:

```
{{
  {
    "crmLeadId": "lead_" + Math.random().toString(36).substring(2, 9),
    "status": "CREATED",
    "email": $json.body.email,
    "phone": $json.body.phone,
    "createdAt": new Date().toISOString()
  }
}}
```

4. Save and activate the workflow. If running locally in n8n, your mock endpoint is: `http://localhost:5678/webhook/mock/crm/leads` (or use Option A / Option B for instant cloud mocks).

Step 2: Configure Trigger & Mock Dirty Leads Generator

1. Create a new workflow named `Topic 3 - Data Validation, Cleaning & Duplicate Prevention`.
2. Add a **Manual Trigger Node** named `Manual_Trigger`.
3. Connect a **Code Node** named `Code_GenerateRawLeads`:
   - **Mode**: `Run Once for All Items`
   - **Language**: `JavaScript`

```
// Simulates 5 marketing form submissions with common data quality flaws
return [
  {
    json: {
      leadId: "LD-001",
      name: "  Sarah Connor ",
      email: "SARAH.CONNOR@SKYNET-RESIST.ORG  ",
      phone: "(555) 234-5678",
      company: "Tech Resistance",
      notes: "Valid lead with US phone formatting"
    }
  },
  {
    json: {
      leadId: "LD-002",
      name: "Bad Email User",
      email: "invalid-email@@nowhere",
      phone: "+1 555 987 6543",
      company: "Test Corp",
      notes: "Malformed email"
    }
  },
  {
```

All rights reserved. Tayana Academy • Confidential Document

```
    json: {
      leadId: "LD-003",
      name: "Spam Bot",
      email: "freebie@mailinator.com",
      phone: "555-111-2222",
      company: "Spam Co",
      notes: "Disposable domain"
    }
  },
  {
    json: {
      leadId: "LD-004",
      name: "Sarah Connor (Duplicate Submission)",
      email: "sarah.connor@skynet-resist.org",
      phone: "+1 (555) 234-5678",
      company: "Tech Resistance Inc",
      notes: "Duplicate of LD-001 with minor whitespace variance"
    }
  },
  {
    json: {
      leadId: "LD-005",
      name: "Marcus Wright",
      email: "marcus@cyberdyne.com",
      phone: "+44 20 7946 0919",
      company: "Cyberdyne Systems",
      notes: "Valid international UK lead"
    }
  }
];
```

Step 3: Implement Sanitization & E.164 Formatting (`Code_SanitizeAndFormat`)

Connect a **Code Node** to standardize phone and email fields:

- **Node Name**: `Code_SanitizeAndFormat`
- **Mode**: `Run Once for Each Item`
- **Language**: `JavaScript`

```
const item = $input.item.json;

// 1. Clean and lowercase email
const cleanEmail = String(item.email || '').trim().toLowerCase();

// 2. Clean phone number and standardize to E.164
let rawPhone = String(item.phone || '').trim();
let cleanPhone = rawPhone.replace(/[^\d+]/g, '');

// Prepend +1 if 10-digit US number without country code
if (/^\d{10}$/.test(cleanPhone)) {
  cleanPhone = '+1' + cleanPhone;
} else if (/^1\d{10}$/.test(cleanPhone)) {
  cleanPhone = '+' + cleanPhone;
}
```

All rights reserved. Tayana Academy • Confidential Document

```
// 3. Trim name and company
const cleanName = String(item.name || '').trim();
const cleanCompany = String(item.company || '').trim();

return {
  json: {
    leadId: item.leadId,
    name: cleanName,
    email: cleanEmail,
    phone: cleanPhone,
    company: cleanCompany,
    rawNotes: item.notes
  }
};
```

Step 4: Implement Lead Quality & Disposable Domain Screening (`Code_ValidateLeadQuality`)

Connect a **Code Node**:

- **Node Name**: `Code_ValidateLeadQuality`
- **Mode**: `Run Once for Each Item`
- **Language**: `JavaScript`

```
const item = $input.item.json;
const errors = [];

// 1. Email format check
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(item.email)) {
  errors.push("INVALID_EMAIL_SYNTAX");
}

// 2. Disposable email domain check
const disposableDomains = ['mailinator.com', 'tempmail.com', '10minutemail.com', 'guerrillamail.com', 'trashmail.com'];
const emailDomain = item.email.split('@')[1] || '';
if (disposableDomains.includes(emailDomain)) {
  errors.push("DISPOSABLE_EMAIL_DOMAIN_BLOCKED");
}

// 3. Phone format check (E.164: + followed by 10 to 15 digits)
const e164Regex = /^\+[1-9]\d{9,14}$/;
if (!e164Regex.test(item.phone)) {
  errors.push("INVALID_E164_PHONE");
}

// 4. Name check
if (!item.name || item.name.length < 2) {
  errors.push("NAME_TOO_SHORT");
}

const isValid = errors.length === 0;

return {
  json: {
    ...item,
    isValid: isValid,
    validationErrors: errors
  }
};
```

All rights reserved. Tayana Academy • Confidential Document

Step 5: Route Quality Leads vs Defective Leads (`IF_IsQualityLead`)

Add an **IF Node**:

- **Node Name**: `IF_IsQualityLead`
- **Condition**: `{{ $json.isValid }}` equals `{{ true }}`

Connect the **False Branch** to a **Code Node** named `Log_QuarantineInvalid`:

```
return {
  json: {
    quarantineType: "SYNTAX_OR_SPAM_VIOLATION",
    leadId: $json.leadId,
    email: $json.email,
    reasons: $json.validationErrors,
    quarantinedAt: new Date().toISOString()
  }
};
```

Step 6: Deduplicate Leads via Fingerprint Hashing (`Code_DeduplicateLeads`)

Connect the **True Branch** of `IF_IsQualityLead` to a **Code Node**:

- **Node Name**: `Code_DeduplicateLeads`
- **Mode**: `Run Once for All Items`
- **Language**: `JavaScript`

```
const items = $input.all();
const seenEmails = new Set();
const seenPhones = new Set();
const results = [];

for (const entry of items) {
  const item = entry.json;
  let isDup = false;
  let dupReason = '';

  if (seenEmails.has(item.email)) {
    isDup = true;
    dupReason = `Duplicate Email: ${item.email}`;
  } else if (seenPhones.has(item.phone)) {
    isDup = true;
    dupReason = `Duplicate Phone: ${item.phone}`;
  } else {
    seenEmails.add(item.email);
    seenPhones.add(item.phone);
  }

  results.push({
    json: {
      ...item,
      isDuplicate: isDup,
      duplicateReason: dupReason
```

All rights reserved. Tayana Academy • Confidential Document

```
    }
  });
}

return results;
```

Step 7: Route Unique Leads vs Duplicates (`IF_IsUniqueLead`)

Add an **IF Node**:

- **Node Name**: `IF_IsUniqueLead`
- **Condition**: `{{ $json.isDuplicate }}` equals `{{ false }}`

Connect the **False Branch** to a **Code Node** named `Log_QuarantineDuplicate`:

```
return {
  json: {
    quarantineType: "DUPLICATE_RECORD_SUPPRESSED",
    leadId: $json.leadId,
    email: $json.email,
    phone: $json.phone,
    reason: $json.duplicateReason,
    quarantinedAt: new Date().toISOString()
  }
};
```

Step 8: Push Clean Unique Lead to Mock CRM (`HTTP_PostCleanLeadToCRM`)

Connect the **True Branch** of `IF_IsUniqueLead` to an **HTTP Request Node**:

- **Node Name**: `HTTP_PostCleanLeadToCRM`
- **Method**: `POST`
- **URL**: `https://dummyjson.com/c/<your-id>` *(from Step 1 Option A, or your Postman Mock URL `https://{{mock-id}}.mock.pstmn.io/crm/v1/leads`)*
- **Send Body**: `ON`
- **Body Content Type**: `JSON`
- **Specify Body**: `Using JSON`
- **JSON**:

```
{{
  {
    "leadId": $json.leadId,
    "fullName": $json.name,
    "email": $json.email,
    "phone": $json.phone,
    "company": $json.company,
    "cleansedAt": new Date().toISOString()
  }
```

All rights reserved. Tayana Academy • Confidential Document

```
}}
```

Step 9: Aggregate Final Run Metrics (`Code_PipelineMetrics`)

Connect to a **Code Node**:

- **Node Name**: `Code_PipelineMetrics`
- **Mode**: `Run Once for All Items`
- **Language**: `JavaScript`

```
const syncedLeads = $input.all();
return [
  {
    json: {
      status: "LEAD_CLEANSING_BATCH_COMPLETE",
      leadsSentToCRM: syncedLeads.length,
      leadRecords: syncedLeads.map(l => ({
        crmLeadId: l.json.crmLeadId,
        email: l.json.email,
        phone: l.json.phone
      })),
      timestamp: new Date().toISOString()
    }
  }
];
```

---

5. Node-by-Node Settings Blueprint

Node ID	Node Name	Node Type	Critical Settings	Custom Expressions / Code
N3-01	`Manual_Trigger`	Manual Trigger	Trigger on demand	N/A
N3-02	`Code_GenerateRawLeads`	Code	Run Once for All Items	Emits 5 mock dirty marketing leads
N3-03	`Code_SanitizeAndFormat`	Code	Run Once for Each Item	E.164 phone conversion, email lowercase
N3-04	`Code_ValidateLeadQuality`	Code	Run Once for Each Item	RFC regex and disposable domain check
N3-05	`IF_IsQualityLead`	IF	Mode: Basic, `isValid == true`	`{{ $json.isValid }} == true`
N3-06	`Log_QuarantineInvalid`	Code	Format quarantine record for syntax/spam	Quarantine log formatter
N3-07	`Code_DeduplicateLeads`	Code	Run Once for All Items	Set-based hash deduplication on email and phone
N3-08	`IF_IsUniqueLead`	IF	Mode: Basic, `isDuplicate == false`	`{{ $json.isDuplicate }} == false`

All rights reserved. Tayana Academy • Confidential Document

N3-09	`Log_QuarantineDuplicate`	Code	Format quarantine log for duplicate	Quarantine log formatter
N3-10	`HTTP_PostCleanLeadToCRM`	HTTP Request	POST to mock CRM endpoint	Clean JSON lead payload
N3-11	`Code_PipelineMetrics`	Code	Run Once for All Items	Summary analytics of processed leads

---

6. Verification Checkpoints & Testing Guide

Test 1: End-to-End Pipeline Verification

1. Ensure your mock CRM endpoint is active (either your DummyJSON Custom Response URL or local mock receiver).
2. Click **Test Workflow** in the `Topic 3` canvas.
3. Observe results across branches:
   - **`Log_QuarantineInvalid`** catches 2 leads:
     - `LD-002` (Reason: `INVALID_EMAIL_SYNTAX`)
     - `LD-003` (Reason: `DISPOSABLE_EMAIL_DOMAIN_BLOCKED`)
   - **`Log_QuarantineDuplicate`** catches 1 lead:
     - `LD-004` (Reason: `Duplicate Email: sarah.connor@skynet-resist.org`)
   - **`HTTP_PostCleanLeadToCRM`** receives exactly 2 clean unique leads:
     - `LD-001` (Sarah Connor, `+15552345678`)
     - `LD-005` (Marcus Wright, `+442079460919`)
4. Confirm `Code_PipelineMetrics` outputs `leadsSentToCRM: 2`.

---

7. Production Risks & Mitigation Strategies

Risk	Impact	Mitigation
Legitimate User with Flagged Domain	False positive rejection of enterprise leads using corporate aliases.	Implement whitelist bypass table for partner domains.
International Number Formatting Errors	Leads from countries with 8-digit or 12-digit schemes misidentified as invalid.	Enforce international country selector in upstream frontend forms.
Deduplication Key Collision	Two distinct contacts sharing a generic office switchboard phone number flagged as duplicates.	Compound deduplication key using both `email` AND `phone` rather than either alone.

All rights reserved. Tayana Academy • Confidential Document